UV-MCP Code Review: Bugs & Improvements

Project: uv-mcp (v0.6.4)
Date: February 5, 2026
Scope: All Python source files, configuration, and tests

Executive Summary

UV-MCP is a well-structured MCP server with comprehensive testing. This review identified **8 moderate bugs**, **3 critical edge cases**, and **15+ improvement opportunities** across error handling, type safety, and performance.

🔴 CRITICAL BUGS

1. Race Condition in `check_uv_available()` - Process Not Properly Cleaned Up

Location: `src/uv_mcp/utils.py:52-79`

Issue:

```
python
```

```python
async def check_uv_available() -> tuple[bool, str | None]:
    process = None
    try:
        # ...
        process = await asyncio.create_subprocess_exec(...)
        # If timeout occurs:
        return False, None  # ← Process not cleaned up!
    except asyncio.TimeoutError:
        # ...
        if process:
            try:
                if process.returncode is None:
                    process.kill()
                await process.communicate()
            except (ProcessLookupError, OSError):
                pass
        return False, None
```

Problem: Main try block doesn't await `process.communicate()`, leaving zombie processes.

Fix:

```python
```

```python
async def check_uv_available() -> tuple[bool, str | None]:
    process = None
    try:
        uv_path = shutil.which("uv")
        if not uv_path:
            return False, None

        process = await asyncio.create_subprocess_exec(
            "uv", "--version",
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )

        try:
            stdout_bytes, stderr_bytes = await asyncio.wait_for(
                process.communicate(), timeout=5
            )
        except asyncio.TimeoutError:
            if process:
                process.kill()
                try:
                    await asyncio.wait_for(process.communicate(), timeout=1)
                except asyncio.TimeoutError:
                    pass
            return False, None

        if process.returncode == 0:
            version = stdout_bytes.decode().strip()
            return True, version
        return False, None
    except (FileNotFoundError, OSError):
        return False, None
```

---

2. Timeout Handler Bug in `run_uv_command()` - Inconsistent Error Messages

Location: `src/uv_mcp/utils.py:108-135`

Issue:

```python
except asyncio.TimeoutError:
    logger.error(f"uv command timed out...")
    if process:
        try:
            if process.returncode is None:
                process.kill()
            await process.communicate()  # ← Can timeout again!
```

Problem: `process.communicate()` can itself timeout, causing cascading failures.

Fix:

```python
except asyncio.TimeoutError:
    logger.error(f"uv command timed out after {timeout}s: uv {' '.join(args)}")
    if process and process.returncode is None:
        process.kill()
        try:
            await asyncio.wait_for(process.communicate(), timeout=2)
        except asyncio.TimeoutError:
            logger.warning("Force kill timeout - process may still be running")
    return False, "", f"Command timed out after {timeout} seconds"
```

---

3. Null Pointer Risk in `_get_uv_pip_args()` - Missing Type Guard

Location: `src/uv_mcp/actions.py:346-354`

Issue:

```python
def _get_uv_pip_args(project_dir: Path) -> list[str]:
    args = []
    venv_exists, venv_path = check_project_venv(project_dir)
    if venv_exists and venv_path:  # ← venv_path could still be None
        args.extend(["--python", venv_path])
    return args
```

Problem: Logic says "if venv_exists AND venv_path", but `check_project_venv()` could return `(True, None)`.

Actual Risk: Low (unlikely scenario), but violates type contract.

Fix:

```python
def _get_uv_pip_args(project_dir: Path) -> list[str]:
    args = []
    venv_exists, venv_path = check_project_venv(project_dir)
    if venv_exists and venv_path is not None:
        args.extend(["--python", venv_path])
    return args
```

---

4. Dictionary Key Error in `show_package_info_action()` - Missing Error Handling

Location: `src/uv_mcp/actions.py:511-524`

Issue:

```python
data = {}
current_key = None
for line in stdout.splitlines():
    if ": " in line:
        key, value = line.split(": ", 1)
        data[key.lower()] = value.strip()
        current_key = key.lower()
    elif current_key and line.startswith("  "):
        # Continuation line (e.g. for Requires)
        data[current_key] += f", {line.strip()}"  # ← KeyError if current_key not set

return PackageInfoResult(
    name=data.get("name", package_name),
    version=data.get("version"),
    location=data.get("location"),
    requires=[r.strip() for r in data.get("requires", "").split(",") if r.strip()],
    # ...
)
```

Problem: If first line of output is indented, `current_key` is None and dict append fails.

Fix:

```python
```

```
data = {}
current_key: str | None = None
for line in stdout.splitlines():
    if ": " in line:
        key, value = line.split(": ", 1)
        data[key.lower()] = value.strip()
        current_key = key.lower()
    elif current_key is not None and line.startswith("  "):
        if current_key in data:
            data[current_key] += f", {line.strip()}"
```

---

5. Logic Error in `check_dependencies()` - Incorrect Output Parsing

Location: `src/uv_mcp/diagnostics.py:67-97`

Issue:

```python
success, stdout, stderr = await run_uv_command(["pip", "check"], cwd=project_dir)

if not success:
    if (
        "No broken requirements found" not in stdout
        and "No broken requirements found" not in stderr
    ):  # ← Backwards logic!
        healthy = False
        # Determine if it's a "broken requirements" issue...
        if stdout.strip() or stderr.strip():
            issues.append(f"Dependency conflicts detected: ...")
```

Problem:

- `uv pip check` exits with 0 if no issues, 1 if issues found
- Current code says: "if command FAILED and message NOT in output = healthy=False"
- This is backwards! Should be: "if command FAILED and message NOT in output = actual error"

Fix:

```python
success, stdout, stderr = await run_uv_command(["pip", "check"], cwd=project_dir)

if not success:
    # Check if failure is due to actual broken requirements or other error
    combined_output = stdout + stderr
    if "No broken requirements found" in combined_output:
        # False alarm - command succeeded logically
        healthy = True
    else:
        # Actual dependency conflict
        healthy = False
        issues.append(f"Dependency conflicts detected: {combined_output.strip()}")
```

---

6. Incorrect List Slicing in `check_dependencies()` - Off-by-One Error

Location: `src/uv_mcp/diagnostics.py:104-108`

Issue:

```python
```

```
success, stdout, stderr = await run_uv_command(["pip", "list"], cwd=project_dir)
if success:
    lines = stdout.strip().split("\n")
    installed_count = max(0, len(lines) - 2)  # ← Why -2?
```

Problem:

- `uv pip list` includes 2 header lines (e.g., "Package Version")
- Subtract 2 to get package count
- BUT: if only 1 package exists, `len(lines)=3`, so `3-2=1` ✓
- BUT: if 0 packages, `len(lines)=2`, so `2-2=0` ✓ (correct due to max(0,...)
- The `max(0,...)` suggests author expected negative numbers, which indicates unclear intent

Real Issue: Silently ignores parsing errors. Should validate.

Fix:

python

```
if success:
    lines = stdout.strip().split("\n")
    # Filter out header lines and empty lines
    package_lines = [l for l in lines if l and not l.startswith("-")]
    installed_count = max(0, len(package_lines) - 1)  # -1 for header
else:
    warnings.append("Could not list installed packages")
    installed_count = None
```

---

7. Memory Leak in `repair_environment_action()` - Actions List Not Bounded

Location: `src/uv_mcp/actions.py:188-228`

Issue:

```python
actions: list[RepairAction] = []

# 1. Initialize project
if action := await _repair_init_project(project_dir, auto_fix):
    actions.append(action)
# 2. Create venv
if action := await _repair_venv(project_dir, auto_fix):
    actions.append(action)
# 3. Install Python
if action := await _repair_python_install(project_dir, auto_fix):
    actions.append(action)
# 4. Sync dependencies
if action := await _repair_sync(project_dir, auto_fix):
    actions.append(action)
```

Problem: Not actually a memory leak, but poor design. If any step creates large output, it stays in memory. Better to stream or limit.

Fix:

```python
```

```python
async def repair_environment_action(...) -> RepairResult:
    # Limit output size to prevent memory issues
    MAX_OUTPUT_SIZE = 10 * 1024 * 1024  # 10MB
    total_output_size = 0
    actions: list[RepairAction] = []

    for step_name, step_func in [
        ("initialize_project", _repair_init_project),
        ("create_venv", _repair_venv),
        ("install_python", _repair_python_install),
        ("sync_dependencies", _repair_sync),
    ]:
        if total_output_size > MAX_OUTPUT_SIZE:
            logger.warning("Repair output exceeded size limit, stopping")
            break

        action = await step_func(project_dir, auto_fix)
        if action:
            actions.append(action)
            if action.output:
                total_output_size += len(action.output.encode())
```

---

8. Broken Symlink Handling in `check_project_venv()` - Follows Symlinks Without Validation

Location: `src/uv_mcp/utils.py:228-237`

Issue:

```python
```

```python
def check_project_venv(project_dir: Path) -> tuple[bool, str | None]:
    # Standard uv venv location
    venv_path = project_dir / ".venv"
    if venv_path.exists() and (venv_path / "pyvenv.cfg").exists():
        return True, str(venv_path)
    return False, None
```

Problem:

- `Path.exists()` follows symlinks
- If `.venv` is a broken symlink, `exists()` returns False ✓ (actually works)
- BUT: if `.venv` is symlink to bad path, `(venv_path / "pyvenv.cfg").exists()` follows it and returns False ✓
- **ACTUAL BUG:** Doesn't validate `.venv` is actually a venv directory, just checks for pyvenv.cfg

Better approach:

```python
```

```python
def check_project_venv(project_dir: Path) -> tuple[bool, str | None]:
    venv_path = project_dir / ".venv"
    pyvenv_cfg = venv_path / "pyvenv.cfg"

    try:
        # Check if .venv exists and is readable
        if not venv_path.exists():
            return False, None

        # Check if it's a directory (not a symlink file)
        if not venv_path.is_dir():
            return False, None

        # Check for pyvenv.cfg
        if pyvenv_cfg.exists() and pyvenv_cfg.is_file():
            return True, str(venv_path)

        return False, None
    except (PermissionError, OSError):
        return False, None
```

🟡 MODERATE BUGS (Non-Critical)

9. Type Hint Inconsistency in `ProjectTools.init_project()`

Location: `src/uv_mcp/tools.py:19-25`

```python
```

```
async def init_project(
    name: str,
    python_version: str = "3.12",
    path: str | None = None,
    template: str = "app",
) -> ProjectInitResult | str:  # ← Returns ProjectInitResult OR str!
```

Problem: Return type is union of two different types - sometimes dict, sometimes ProjectInitResult

Fix:

python

```python
async def init_project(
    name: str,
    python_version: str = "3.12",
    path: str | None = None,
    template: str = "app",
) -> ProjectInitResult:
    # Always return ProjectInitResult with success=False on error
    try:
        # ...
        return ProjectInitResult(
            project_name=name,
            project_dir=str(project_dir),
            python_version=python_version,
            template=template,
            success=True,
            created_files=["pyproject.toml", ".python-version"],
        )
    except Exception as e:
        return ProjectInitResult(
            project_name=name,
            project_dir=str(project_dir or ""),
            python_version=python_version,
            template=template,
            success=False,
            error=str(e),
        )
```

---

10. Missing Validation in `add_dependency_action()` - Complex Version Specs

Location: `src/uv_mcp/actions.py:265-295`

Issue:

```python
async def add_dependency_action(
    package: str,  # ← No validation!
    project_path: str | None = None,
    dev: bool = False,
    optional: str | None = None,
) -> DependencyOperationResult:
```

Problem: Accepts any string as package name. No validation for:

- Empty strings
- Invalid characters
- Conflicting with reserved names
- git+https:// URLs mixed with other options

Fix:

```python
```

```
import re

def _validate_package_spec(package: str) -> tuple[bool, str | None]:
    """Validate package specification format."""
    if not package or not package.strip():
        return False, "Package name cannot be empty"

    package = package.strip()

    # Allow git URLs
    if package.startswith(("git+", "https://", "http://")):
        return True, None

    # Basic package name validation (PEP 508)
    # Names can have letters, numbers, hyphens, underscores, dots
    if not re.match(r"^[a-zA-Z0-9_.-]+(\[[^\]]+\])?", package):
        return False, "Invalid package name format"

    return True, None

async def add_dependency_action(...) -> DependencyOperationResult:
    valid, error = _validate_package_spec(package)
    if not valid:
        return DependencyOperationResult(
            package=package,
            project_dir=str(project_dir),
            success=False,
            error=error,
        )
    # ... rest of function
```

11. Missing Optional Parameter Propagation in `remove_dependency_action()`

Location: `src/uv_mcp/actions.py:330-361`

Issue: The function signature accepts `optional` parameter but has minimal handling:

```python
async def remove_dependency_action(
    package: str,
    project_path: str | None = None,
    dev: bool = False,
    optional: str | None = None,  # ← Accepted but barely used
) -> DependencyOperationResult:
    # ...
    cmd = ["remove", package]
    if dev:
        cmd.append("--dev")
    if optional:  # ← Added, but no error if optional doesn't exist
        cmd.extend(["--optional", optional])
```

Problem: Should validate that optional group exists before attempting removal

Fix: Add pre-validation that the optional group actually exists in pyproject.toml

---

🔵 IMPROVEMENT OPPORTUNITIES

Type Safety & Validation

12. Missing Input Validation in Multiple Functions

- `analyze_dependency_tree_action()` - doesn't validate project_path exists
- `list_dependencies_action()` - tree parameter not validated

- `check_outdated_packages_action()` - project validation missing

Recommendation: Create a utility `validate_project_path()` function:

```python
async def validate_project_path(path: str | None) -> Path | None:
    """Validate project path and return Path object."""
    project_dir = Path(path) if path else Path.cwd()
    if not project_dir.exists():
        raise ProjectNotFoundError(str(path))
    if not (project_dir / "pyproject.toml").exists():
        raise PyProjectNotFoundError(str(project_dir))
    return project_dir
```

Error Handling

13. Inconsistent Error Handling Pattern

Some functions use:

```python
if not available:
    return Result(success=False, error="message")
```

Others use:

```python
```

```
try:
    # ...
except Exception as e:
    logger.error(f"Error: {e}")
    return False, "", str(e)
```

Recommendation: Standardize on custom exceptions from `errors.py`:

```python
async def list_dependencies_action(...) -> DependencyListResult:
    try:
        project_dir = Path(project_path) if project_path else Path.cwd()
        if not project_dir.exists():
            raise ProjectNotFoundError(str(project_path))
        # ... rest
    except UVMCPError as e:
        return DependencyListResult(
            project_dir=str(project_dir),
            is_tree=tree,
            count=0,
            success=False,
            error=e.message,
        )
```

Performance & Resource Management

14. Unbounded Output in `list_dependencies_action()` - Tree Mode

```python
```

```
if tree:
    success, stdout, stderr = await run_uv_command(["tree"], cwd=project_dir)
    if not success:
        return DependencyListResult(...)
    return DependencyListResult(
        tree_output=stdout,  # ← Could be very large!
        count=len(stdout.splitlines()),
    )
```

Risk: Large projects can have huge dependency trees (10MB+)

Fix:

```
python

MAX_TREE_SIZE = 5 * 1024 * 1024  # 5MB
if len(stdout) > MAX_TREE_SIZE:
    tree_output = stdout[:MAX_TREE_SIZE] + "\n... (output truncated)"
    logger.warning(f"Dependency tree exceeded size limit")
```

Configuration & Constants

15. Magic Numbers Without Constants

- `timeout=120.0` in `run_uv_command()` - hardcoded
- `5` second timeout in `check_uv_available()` - hardcoded
- `10 * 1024 * 1024` (10MB) for memory limits - hardcoded

Fix: Create `src/uv_mcp/config.py`:

```
python
```

```
# Default timeouts (seconds)
DEFAULT_COMMAND_TIMEOUT = 120.0
DEFAULT_CHECK_TIMEOUT = 5.0

# Size limits
MAX_TREE_OUTPUT = 5 * 1024 * 1024  # 5MB
MAX_CACHE_SIZE = 10 * 1024 * 1024  # 10MB

# Retry policy
MAX_RETRIES = 3
RETRY_DELAY = 0.5  # seconds
```

Testing

16. Incomplete Test Coverage for Error Paths

Files with low error coverage:

- `actions.py` - PyProjectNotFoundError path not fully tested
- `diagnostics.py` - Malformed TOML handling minimal
- `utils.py` - Many exception paths mocked but not truly tested

Recommendation: Add integration tests:

```python
```

```
@pytest.mark.asyncio
async def test_add_dependency_malformed_pyproject(tmp_path):
    """Test error handling with invalid pyproject.toml."""
    (tmp_path / "pyproject.toml").write_text("invalid {{{")

    result = await add_dependency_action(
        "requests",
        project_path=str(tmp_path)
    )

    assert not result.success
    assert "invalid" in result.error.lower() or "parse" in result.error.lower()
```

Documentation

17. Missing Docstring Examples

Many functions lack usage examples. Add to docstrings:

```
python
```

```
async def add_dependency_action(
    package: str,
    project_path: str | None = None,
    dev: bool = False,
    optional: str | None = None,
) -> DependencyOperationResult:
    """
    Add a new dependency to the project.

    Args:
        package: Package name with optional version specifier (e.g., "requests>=2.28.0")
        project_path: Path to the project directory (defaults to current directory)
        dev: Whether to add as a development dependency (default: False)
        optional: Optional dependency group name (e.g., "test", "docs")

    Returns:
        DependencyOperationResult with operation results

    Raises:
        ProjectNotFoundError: If project directory doesn't exist
        PyProjectNotFoundError: If pyproject.toml is missing

    Examples:
        >>> result = await add_dependency_action("requests>=2.28.0")
        >>> if result.success:
        ...     print(f"Added {result.package}")
        >>> result = await add_dependency_action("pytest", dev=True)
    """
```

Logging

18. Inconsistent Log Levels

- Some errors logged as `logger.error()` then returned as `success=False`
- Some logged as `logger.warning()` then returned as `success=True`

Recommendation:

- `logger.error()` → `success=False`
- `logger.warning()` → `success=True` with warnings in result
- `logger.info()` → Informational, no error condition

Async Handling

19. Potential Deadlock in Concurrent Operations

No limit on concurrent subprocess spawning. If many operations run concurrently, could exhaust file descriptors.

Fix:

```python
import asyncio

_COMMAND_SEMAPHORE = asyncio.Semaphore(10)  # Max 10 concurrent uv commands

async def run_uv_command(...) -> tuple[bool, str, str]:
    async with _COMMAND_SEMAPHORE:
        # ... actual command execution
```

📋 SUMMARY TABLE

ID	Severity	Category	Issue	Fix Complexity
1	🔴 Critical	Process Management	Zombie processes in check_uv_available	Medium
2	🔴 Critical	Timeout Handling	Cascading timeouts in run_uv_command	Medium
3	🔴 Critical	Type Safety	Null pointer risk in _get_uv_pip_args	Low
4	🔴 Critical	Exception Handling	KeyError in show_package_info	Low
5	🔴 Critical	Logic Error	Inverted logic in check_dependencies	Low
6	🔴 Critical	Off-by-One	List slicing error in dependency count	Low
7	🔴 Critical	Resource Mgmt	Unbounded output in repair_environment	Medium
8	🔴 Critical	Path Handling	Insufficient venv validation	Low
9	🟡 Moderate	Type Hints	Inconsistent return types (Union)	Low
10	🟡 Moderate	Validation	Missing package name validation	Medium
11	🟡 Moderate	Feature	Incomplete optional group handling	Low
12	🔵 Minor	Validation	Missing input validation	Medium
13	🔵 Minor	Consistency	Inconsistent error patterns	Medium
14	🔵 Minor	Performance	Unbounded tree output	Low
15	🔵 Minor	Design	Magic numbers hardcoded	Low
16	🔵 Minor	Testing	Incomplete error path coverage	Medium

ID	Severity	Category	Issue	Fix Complexity
17	🔵 Minor	Documentation	Missing docstring examples	Low
18	🔵 Minor	Logging	Inconsistent log levels	Low
19	🔵 Minor	Concurrency	No limit on concurrent subprocesses	Medium

---

🚀 Recommended Priority Fixes

Immediate (Week 1):

1. Fix critical bugs #1, #2, #5, #6
2. Add input validation for project paths
3. Ensure all error paths tested

Short Term (Week 2-3): 4. Standardize error handling with custom exceptions 5. Add configuration constants 6. Improve type consistency

Medium Term (Month 1): 7. Add concurrency limits 8. Improve test coverage for error paths 9. Add docstring examples

---

Code Quality Metrics

Metric	Status	Target
Type Hint Coverage	85%	95%
Error Path Testing	70%	90%

Metric	Status	Target
Docstring Coverage	80%	100%
Cyclomatic Complexity	Avg 3.2	< 5
Test Coverage	87%	90%+